

Week13_예습과제_백재은

08_06 토픽 모델링 - 20 뉴스그룹

토픽 모델링: 문서 집합에 숨어 있는 주제를 찾아내는 것

머신러닝 기반 토픽 모델 ;> 숨겨진 주제를 효과적으로 표현할 수 있는 중심 단어를 함축적으로 추출

자주 사용되는 기법: LSA, LDA

LDA를 활용한 토픽 모델링

```
['alt.atheism', 'comp.graphics', 'comp.os.ms-windows.misc', 'comp.sys.ibm.pc.hardware',  
'comp.sys.mac.hardware', 'comp.windows.x', 'misc.forsale', 'rec.autos', 'rec.motorcycles',  
'rec.sport.baseball', 'rec.sport.hockey', 'sci.crypt', 'sci.electronics', 'sci.med', 'sci.space',  
'soc.religion.christian', 'talk.politics.guns', 'talk.politics.mideast', 'talk.politics.misc',  
'talk.religion.misc']
```

-> 20 뉴스그룹 데이터 세트 적용

사이킷런 LDA 기반 토픽 모델링 -> LatentDirichletAllocation 클래스 제공

```
from sklearn.datasets import fetch_20newsgroups  
from sklearn.feature_extraction.text import CountVectorizer  
from sklearn.decomposition import LatentDirichletAllocation  
  
# 모터사이클, 야구, 그래픽스, 윈도우즈, 운동, 기독교, 전자공학, 의학 8개 주제를 추출.  
cats = ['rec.motorcycles', 'rec.sport.baseball', 'comp.graphics', 'comp.windows.x',  
        'talk.politics.mideast', 'soc.religion.christian', 'sci.electronics', 'sci.med']  
  
# 위에서 cats 변수로 기재된 카테고리만 추출. fetch_20newsgroups()의 categories에 cats 입력  
news_df = fetch_20newsgroups(subset='all', remove=('headers', 'footers', 'quotes'),  
                             categories=cats, random_state=0)  
  
# LDA는 Count 기반의 벡터화만 적용합니다.  
count_vect = CountVectorizer(max_df=0.95, max_features=1000, min_df=2, stop_words='english',  
                             ngram_range=(1, 2))
```

```
feat_vect = count_vect.fit_transform(news_df.data)  
print('CountVectorizer Shape:', feat_vect.shape)
```

-> fetch_20newsgroups() APO:
categories 파라미터 이용 필요 주제 필터링
후 추출 -> 추출 텍스트 Count 기반 벡터화
변환

-> 객체 변수 feat_vect: 7862doc, 1000feat

```
lda = LatentDirichletAllocation(n_components=8, random_state=0)  
lda.fit(feat_vect)
```

-> LDA 토픽 모델링 수행

-> 토픽 개수 8개(n_components)

-> 속성값 확인(print(lda.components_.shape): (8, 1000) 출력

=> 8개의 토픽별로 1000개의 word피처가 해당 토픽별로 연관도 값 가짐

```
def display_topics(model, feature_names, no_top_words):
    for topic_index, topic in enumerate(model.components_):
        print("Topic #", topic_index)

        # components_ array에서 가장 값이 큰 순으로 정렬했을 때, 그 값의 array 인덱스를 반환.
        topic_word_indexes = topic.argsort()[::-1]
        top_indexes=topic_word_indexes[:no_top_words]

        # top_indexes대상이 인덱스별로 feature_names에 해당하는 word feature 추출 후 join으로 concat
        feature_concat = ' '.join([feature_names[i] for i in top_indexes])
        print(feature_concat)

# CountVectorizer 객체 내의 전체 word의 명칭을 get_feature_names()를 통해 추출
feature_names = count_vect.get_feature_names()

# 토픽별 가장 연관도가 높은 word를 15개만 추출
display_topics(lda, feature_names, 15)
```

-> display_topics() 함수 생성: 각 토픽별로
연관도가 높은 순으로 Word 나열

=> OUTCOME

Topic #0: 불분명한 주제어 유, 주로 의학 관련

Topic #1: 명확X, 일반적인 단어

Topic #2: 컴퓨터 그래픽스 영역 주제어 다수 포함

Topic #3: 일반적인 단어

Topic #4: 명확한 중동 영역 주제어 추출

Topic #5: 일부 컴퓨터 그래픽스 포함, 전반적인 컴퓨터 용어 — 따라서 매핑 hard

Topic #6: 명확한 기독교 관련 주제어 추출

Topic #7: 윈도우 운영체제 관련 주제어 추출

08_07 문서 군집화 소개와 실습

a. 개념

문서 군집화: 비슷한 텍스트 구성의 문서를 군집화하는 것, 기존 텍스트 분류 기반의 문서 분류와 유사

-> 차이점: 학습 데이터 세트가 필요 없는 비지도학습 기반 동작

b. Opinion Review 데이터 세트를 이용한 문서 군집화 수행

1) 데이터 세트 소개

-> 51개 텍스트 파일, 호텔/자동차/전자제품 사이트에서 가져온 리뷰 문서, 100개 정도의 문장 각각 포함

2) for문을 활용해 파일을 DF로 로딩

[Output]

	filename	opinion_text
0	accuracy_gamer_nova_201601_gps	. and is very, very accurate. v0 but for the most part, we find that the Garmin software provides accurate directions, whenever we intend to go. v0 This functionality is very useful.
1	bathroom_beswestern_hotel_its	The room was not overly big, but clean and very comfortable beds, a great shower and very clean bathrooms. v0 The second room was smaller, with a very convenient bathroom layout, but at least it was quarter and we were able to sleep. v0
2	battery_life_amazon_kindle	After I plugged it in to my USB hub on my computer to charge the battery the charging cord design is very clever first. After you have paged to a 500 page book one page at a time to get from Chapter 2 to Chapter 15, see how excited you are about a low battery and all the time it took to get there first.
3	battery_life_sony_nexo_8ph	short battery life I moved up from an 8ph. v0 I love this spot except for the battery life. v0
4	battery_life_nexus_1000ha	60Hz 5.33P580 (cpu: ghosy dhaip), 3. Cof 220W L.L. on Battery, and a 1. v0 Not to mention that as of now...

-> 2의 결과 (document_df.head())

3) TF-IDF 형태로 피쳐 벡터화

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf_vect = TfidfVectorizer(tokenizer=LemNormalize, stop_words='english', \
                             ngram_range=(1,2), min_df=0.05, max_df=0.85)
```

-> LemNormalize() 함수 생성

-> n_gram: (1, 2) / min_df, max_df 범위 설정

```
#opinion_text 칼럼값으로 feature vectorization 수행
feature_vect = tfidf_vect.fit_transform(document_df['opinion_text'])
```

-> TfidfVectorizer의 fit_transform() 인자로 document_df DF의 opinion_text 칼럼 입력 시 개별 문서 텍스트에 대해 TF-IDF 변환된 피쳐 벡터화된 행렬 아웃풋

4) 군집화 수행

```
from sklearn.cluster import KMeans

# 5개 집합으로 군집화 수행, 예제를 위해 동일한 클러스터링 결과 도출을 random_state=0
km_cluster = KMeans(n_clusters=5, max_iter=10000, random_state=0)
km_cluster.fit(feature_vect)
cluster_label = km_cluster.labels_
cluster_centers = km_cluster.cluster_centers_
```

-> K-평균 적용

-> 5개 Centroid 기반 군집화 확인, max_iter = 10000

5) document_df.head() 를 통해 각 주제별로 유사한 형태로 구성되었는지 확인

6) sort_values를 통해 document_df DataFrame 객체에서 cluster_label이 어떤 파일명으로 매칭되었는지 보고 군집화 결과 확인 (cluster_label = 0 ~4 데이터 세트)

	filename	opinion_text	cluster_label
1	bathroom_beswestern_hotel_its	The room was not overly big, but clean and very comfortable beds, a great shower and very clean bathrooms. v0 The second room was smaller, with a very convenient bathroom layout, but at least it was quarter and we were able to sleep. v0	0
22	room_holiday_m_hendon	We arrived at 23.30 hours and they could not recommend a restaurant so we decided to go to Tesco, with very limited choices but when you are hungry you do not care! Next day they rang the bell at 8.00 hours to clean the room, not doing very nice being woken up so early! Every day they gave us.	0
30	rooms_beswestern_hotel_its	Great Location - Nice Rooms - H.	0
31	rooms_sawestel_chicago	The Sawestel is one of our favorite hotels in Chicago and the corner rooms have the most fantastic views in the city. v0 The rooms look like they were just remodeled and upgraded, there was an HD TV and a nice shower. Nothing to put my phone in I could not get the alarm to wake up with my music instead of the radio. v0	0

-> Cluster #0: 호텔에 대한 리뷰로 군집화

-> 0 ~4 데이터 세트 군집화 결과: 군집 개수 too much -> 세분화 경향 -> 3개 그룹으로!

7) 5개 그룹 -> 3개 그룹으로 군집화

```
from sklearn.cluster import KMeans

# 3개의 집합으로 군집화
km_cluster = KMeans(n_clusters=3, max_iter=10000, random_state=0)
km_cluster.fit(feature_vect)
cluster_label = km_cluster.labels_

# 소속 클러스터를 cluster_label 칼럼으로 할당하고 cluster_label 값으로 정렬
document_df['cluster_label'] = cluster_label
document_df.sort_values(by='cluster_label')
```

-> 결과: Cluster #0 포터블 전자기기 리뷰

Cluster #1: 자동차 리뷰

Cluster #2: 호텔 리뷰 로 군집화 완

c. 군집별 핵심 단어 추출하기

1) 각 군집을 구성하는 핵심 단어 확인

KMeans 객체: 각 군집을 구성하는 단어 피처가 군집의 중심을 기준으로 얼마나 가까이 위치해 있는지 `clusters_centers_` 라는 속성으로 제공 -> 배열 값, 행 = 개별 군집, 열 = 개별 피처

[Output]

```
cluster_centers shape : (3, 4611)
[[0.01005322 0.         ... 0.00706287 0.         ]
 [0.         0.00092551 0.         ... 0.         0.         ]
 [0.         0.00099499 0.00174637 ... 0.         0.00183397 0.00144581]]
```

-> `clusters_centers_`: 군집 3개, word 피처 4611개 구성 (군집 3개로 생성한 KMeans 객체로 수행)

-> 배열 값: 0 ~ 1, 1에 가까울수록 중심

2) `clusters_centers` 속성값을 이용해 각 군집별 핵심 단어 추출

`clusters_centers_` 속성: 넘파이의 `ndarray` / `ndarray`의 `argsort()`[:, ::-1] 이용하여 배열 내 값이 큰 순으로 정렬된 위치 인덱스 값 반환 (핵심 단어 피처의 이름 출력 목적)

`get_clster_deatails()` 함수 생성: 위에 대한 처리 담당

-> 위치 인덱스 추출: 해당 인덱스 이용 핵심 단어 이름과 상대 위치 값 추출, `cluster_details` 라는 Dict 객체 변수에 기록하고 반환

3) `get_clster_deatails()` 호출 -> dictionary 를 원소로 가지는 리스트 `cluster_details` 반환

-> 개별 군집 번호, 핵심 단어, 핵심 단어 중심 위치 상댓값, 파일명 속성 값 정보

4) `print_cluster_details()` 함수: 위 사항 보기 좋게 표현

5) `get_clster_deatails()`, `print_cluster_details()` 함수 호출

```
feature_names = tfidf_vect.get_feature_names()

cluster_details = get_cluster_details(cluster_model=km_cluster, cluster_data=document_df,
                                     feature_names=feature_names, clusters_num=3, top_n_features=10 )
print_cluster_details(cluster_details)
```

-> 인자: KMeans 군집화 객체, 파일명 추출을 위한 `document_df` DF, 핵심 단어 추출을 위한 피처명 리스트, 전체 군집 개수, 핵심 단어 추출

-> 피처명 리스트: TF-IDF 변환된 객체에서 `get_feature_names()` 로 추출

-> Cluster #0: 화면과 배터리 수명 등 핵심 단어로 군집화 - 포터블 전자제품 군집

-> Cluster #1: 실내 인테리어, 좌석, 연료 효율 등 핵심 단어로 군집화 - 자동차 군집

-> Cluster #2: 방과 서비스 핵심 단어로 군집화 - 호텔 군집

08_08 문서 유사도

a. 문서 유사도 측정 방법 - 코사인 유사도

코사인 유사도: 문서와 문서 간 유사도 비교 시 주로 사용, 벡터와 벡터 간의 유사도 비교할 때 벡터의 상호 방향성이 얼마나 유사한지 → 두 벡터 사이의 사잇각 구해서 수치로 적용

b. 두 벡터 사잇각



$$A * B = \|A\| \|B\| \cos \theta$$

사잇각에 따른 상호관계

두 벡터의 내적 = 두 벡터 크기 곱 * 코사인 각도 곱

- 유사도 수식

$$\text{similarity} = \cos \theta = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

→ 두 벡터의 내적을 총 벡터 크기 합으로 나눈 것

→ 내적 결과를 총 벡터 크기로 L2 Norm

- 많이 사용되는 이유

문서를 피쳐 벡터화 변환 시 차원이 많은 희소행렬이 되기 쉬움 → 문서와 문서 벡터 간의 크기에 기반한 유사도 지표의 정확도 떨어짐

문서가 긴 경우 → 단어의 빈도수가 높을 가능성이 높아 공정한 비교 불가

1) 문서 유사도를 코사인 유사도 기반으로 확인

```
import numpy as np

def cos_similarity(v1, v2):
    dot_product = np.dot(v1, v2)
    l2_norm = (np.sqrt(sum(np.square(v1))) * np.sqrt(sum(np.square(v2))))
    similarity = dot_product / l2_norm

    return similarity
```

-cos_similarity() 함수 생성

2) 문서를 TF-IDF로 벡터화된 행렬로 변환

```
from sklearn.feature_extraction.text import TfidfVectorizer

doc_list = ['if you take the blue pill, the story ends',
            'if you take the red pill, you stay in Wonderland',
            'if you take the red pill, I show you how deep the rabbit hole goes']

tfidf_vect_simple = TfidfVectorizer()
feature_vect_simple = tfidf_vect_simple.fit_transform(doc_list)
print(feature_vect_simple.shape)
```


3) 밀집 행렬로 변환 후 다시 각각을 배열로 변환

```
# TfidfVectorizer로 transform()한 결과는 희소 행렬이므로 밀집 행렬로 변환.
feature_vect_dense = feature_vect_simple.todense()

# 첫 번째 문장과 두 번째 문장의 피쳐 벡터 추출
vect1 = np.array(feature_vect_dense[0]).reshape(-1, )
vect2 = np.array(feature_vect_dense[1]).reshape(-1, )

# 첫 번째 문장과 두 번째 문장의 피쳐 벡터로 두 개 문장의 코사인 유사도 추출
similarity_simple = cos_similarity(vect1, vect2)
print('문장 1, 문장 2 Cosine 유사도: {0:.3f}'.format(similarity_simple))
```

-> feature_vect_dense[n]: doc_list n+1 번째 문서의 피쳐 벡터화

-> cos_similarity 호출, 두 문서의 유사도 측정 (0.402)

4) 코사인 유사도 측정 - sklearn.metrics.pairwise.cosine_similarity API

1st 파라미터: 비교 기준이 되는 문서의 피쳐 행렬 / 2nd 파라미터: 비교되는 문서의 피쳐 행렬

cosine_similarity: 희소 행렬, 밀집 행렬 가능 / 행렬, 배열 가능 -> 변환 필요 X

```
from sklearn.metrics.pairwise import cosine_similarity

similarity_simple_pair = cosine_similarity(feature_vect_simple[0], feature_vect_simple)
print(similarity_simple_pair)
```

[Output]

```
[[1. 0.40207758 0.40425045]]
```

-> 첫 번째 문서 vs 두 번째, 세 번째 문서 유사도 측정

-> 1(자기 자신 비교) 값 제외 시 feature_vect[1:] 사용

5) 쌍으로 코사인 유사도 값 계산

```
similarity_simple_pair = cosine_similarity(feature_vect_simple, feature_vect_simple)
print(similarity_simple_pair)
print('shape:', similarity_simple_pair.shape)
```

[Output]

```
[[1. 0.40207758 0.40425045]
 [0.40207758 1. 0.45647296]
 [0.40425045 0.45647296 1. ]]
shape: (3, 3)
```

-> (3, 3) ndarray 형태

-> 1 row: 1 vs 2, 3

-> 2 row: 2 vs 1, 3

-> 3 row: 3 vs 1, 2

C. Opinion Review 데이터 세트 이용한 문서 유사도 측정

1) 데이터 세트를 다시 DF로 로드, 문서 군집화 적용

-> 호텔을 주제로 군집화된 문서를 이용해 특정 문서와 다른 문서 간의 유사도 측정

-> 피쳐 벡터화된 데이터를 cosine_similarity()를 이용해 상호 비교

2) 호텔 주제 군집화 데이터 추출

```
from sklearn.metrics.pairwise import cosine_similarity

# cluster_label=2인 데이터는 호텔로 군집화된 데이터임. DataFrame에서 해당 인덱스를 추출
hotel_indexes = document_df[document_df['cluster_label']==2].index
print('호텔로 클러스터링 된 문서들의 DataFrame Index:', hotel_indexes)

# 호텔로 군집화된 데이터 중 첫 번째 문서를 추출해 파일명 표시.
comparison_docname = document_df.iloc[hotel_indexes[0]]['filename']
print('#### 비교 기준 문서명 ', comparison_docname, ' 와 타 문서 유사도####')

''' document_df에서 추출한 Index 객체를 feature_vect로 입력해 호텔 군집화된 feature_vect 추출
이를 이용해 호텔로 군집화된 문서 중 첫 번째 문서와 다른 문서 간의 코사인 유사도 측정. '''
similarity_pair = cosine_similarity(feature_vect[hotel_indexes[0]], feature_vect[hotel_indexes])
print(similarity_pair)
```

[Output]

```
호텔로 클러스터링 된 문서들의 DataFrame Index: Int64Index([1, 13, 14, 15, 20, 21, 24, 28, 30, 31, 32, 36, 39, 40, 45, 46], dtype='int64')
#### 비교 기준 문서명 bathroom.bestwestern_hotel_sfo 와 타 문서 유사도####
[[1. 0.04306888 0.05221059 0.06189595 0.95846178 0.06193118
 0.03638665 0.11742762 0.38938865 0.32619948 0.51442299 0.11282857
 0.13080623 0.1386783 0.09518068 0.07049362]]
```

-> TfidfVectorizer 데이터 추출

-> 객체 변수 document_df 에서 호텔로 군집화된 문서의 인덱스 추출

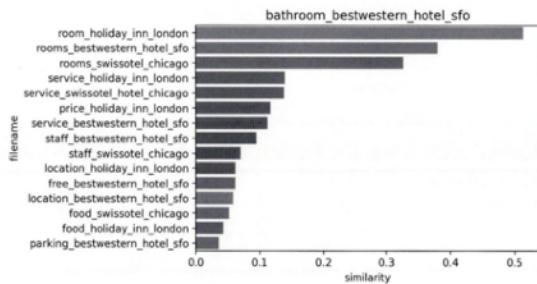
-> feature_vect 에서 문서의 피쳐 벡터 추출

3) 첫 번째 문서와 다른 문서 간 유사도가 높은 순으로 정렬, 시각화

-> ndarray 반환 값을 판다스 인덱스로 이용하기 위해 reshape(-1)로 차원 변경

[Output]

```
가장 유사도가 큰 파일명 및 유사도:
filename    room_holiday_inn_london
similarity      0.514423
```



-> 첫 번째 문서(샌프란시스코의 베스트 위스턴 호텔 화장실 리뷰)와 가장 유사한 문서 = room_holiday_inn_london

-> 0.514 코사인 유사도 값

08_09 한글 텍스트 처리 - 네이버 영화 평점 분석

a. 한글 NLP 처리의 어려움

- 띄어쓰기와 다양한 조사 -> 의미 왜곡 발생 쉬움

b. KoNLPy 소개

- 파이썬의 한글 형태소 패키지

- 형태소 분석: 말뭉치를 형태소 어근 단위로 쪼개고 각 형태소에 품사 태깅을 부착하는 작업

- 설치 과정 설명~

c. 데이터 로딩

```
import pandas as pd

train_df = pd.read_csv('ratings_train.txt', sep='\t', encoding='cp949')
train_df.head(3)
```

[Output]

	id	document	label
0	9976970	대 덕방. 진짜 짜증나네요 목소리	0
1	3819312	음... 포스터보고 조감독하듯... 오버연기조차 가쉽지 않구나	1
2	10265843	너무재밌었다그래서보는것을추천한다	0

-네이버 평점 데이터

-Ratings_train.txt 파일 DF로 로딩, 데이터 살펴보기

-> \t로 칼럼 분리, sep 파라미터 탭으로 설명하여 DF 생성 + 인코딩 설정

2) 0과 1 Label 값 비율 확인

```
train_df['label'].value_counts()
```

[Output]

```
0    75173
1    74827
```

-> 균등 분포

->

3) 데이터 가공

-> 정규 표현식 이용, 숫자를 공백으로 변경

-> 테스트 데이터 세트 로딩 후 Null 및 숫자를 공백으로 변환

-> id 칼럼 삭제

4) Twitter 클래스를 이용한 한글 형태소 분석 -> 형태소 단어로 토큰화

```
from konipy.tag import Twitter

twitter = Twitter()

def tw_tokenizer(text):
    # 입력 인자로 들어온 텍스트를 형태소 단어로 토큰화해 리스트 형태로 반환
    tokens_ko = twitter.morphs(text)
    return tokens_ko
```

-> morphs() 메서드 이용: 입력 인자로 들어온 문장 형태소 단어 형태로 토큰화 -> list 객체로 반환

-> tw_tokenizer() 함수 생성: 반환용...

5) TF-IDF 피쳐 모델 생성

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV

# Twitter 객체의 morphs() 객체를 이용한 tokenizer를 사용. ngram_range는 (1, 2)
tfidf_vect = TfidfVectorizer(tokenizer=tw_tokenizer, ngram_range=(1, 2), min_df=3, max_df=0.9)
tfidf_vect.fit(train_df['document'])
tfidf_matrix_train = tfidf_vect.transform(train_df['document'])
```

-> tokenizer: tw_tokenizer() 함수 /
ngram = (1, 2) / max_df = 3, 상위 90% 제한

6) 로지스틱 회귀를 이용해 분류 기반 감성 분석 수행

```
# 로지스틱 회귀를 이용해 감성 분석 분류 수행.
lg_clf = LogisticRegression(random_state=0, solver='liblinear')

# 파라미터 C 최적화를 위해 GridSearchCV를 이용.
params = {'C': [1, 3.5, 4.5, 5.5, 10]}
grid_cv = GridSearchCV(lg_clf, param_grid=params, cv=3, scoring='accuracy', verbose=1)
grid_cv.fit(tfidf_matrix_train, train_df['label'])
print(grid_cv.best_params_, round(grid_cv.best_score_, 4))
```

[Output]

{'C': 3.5} 0.8593

-> 하이퍼 파라미터 C의 최적화:
GridSearchCV 이용

-> C = 3.5 일 때 최고 0.8593의 정확도

7) 최종 감성 분석 예측 수행

```
from sklearn.metrics import accuracy_score

# 학습 데이터를 적용한 TfidfVectorizer를 이용해 테스트 데이터를 TF-IDF 값으로 피쳐 변환함.
tfidf_matrix_test = tfidf_vect.transform(test_df['document'])

# classifier는 GridSearchCV에서 최적 파라미터로 학습된 classifier를 그대로 이용
best_estimator = grid_cv.best_estimator_
preds = best_estimator.predict(tfidf_matrix_test)

print('Logistic Regression 정확도: ', accuracy_score(test_df['label'], preds))
```

[Output]

Logistic Regression 정확도: 0.86174

-> 학습 시 적용한 TfidfVectorizer 사용

-> 객체 변수 tfidf_vect 이용 -> 테스트 데이터 document 칼럼에 transform() 수행